ADR-001：采用前后端分离的模块化单体架构

状态

已接受

背景

Git Guild 是一个 4 人学生团队在 10 周内交付的课程项目。系统既包含任务协作、推荐匹配、审核反馈等高业务耦合流程，也需要与外部代码托管底座集成。团队需要在交付速度、可维护性和后续演进之间做平衡。

决策

我们决定采用前后端分离的模块化单体架构：

- 前端使用 Vue 3 单页应用
- 后端使用 Spring Boot 3 单体应用
- 后端内部按领域模块划分职责边界

理由

- 微服务会显著增加服务拆分、通信、部署和调试复杂度，不适合当前团队规模
- 任务流转、审核、推荐和成长反馈之间耦合紧密，放在单体内更易保持一致性
- 前后端分离有利于并行开发与后续扩展
- 模块化单体既能保证当前开发效率，也为后续拆分保留演进空间

后果

- 正面影响：实现路径清晰，部署简单，适合课程项目快速落地
- 负面影响：所有业务集中在一个后端应用中，后期复杂度上升时需要更强的模块治理
- 需要关注的风险：如果模块边界定义不严格，单体容易演变为高耦合结构

AI 辅助记录

- AI 初稿内容摘要：AI 在无强约束场景下倾向推荐事件驱动微服务，强调解耦和可扩展性
- 人工修订内容：团队结合周期、人力和实现阶段，将方案收敛为“前后端分离 + 模块化单体”
- 修订理由：AI 高估了课程项目中微服务带来的现实收益，低估了其部署与协作成本

AI 初稿与人工修订对比

- 采用事件驱动的微服务架构，每个核心能力独立部署
- 通过消息队列连接任务、成长、通知等服务
+ 采用前后端分离的模块化单体架构
+ 在单体内部通过模块边界和应用内事件实现解耦
+ 暂不引入跨服务部署与消息队列基础设施